

Bài 18: JDBC Cơ Bản

JDBC (Java Database Connectivity) là API cho phép ứng dụng Java kết nối và tương tác với cơ sở dữ liệu.

1. Kiến trúc JDBC



- JDBC API cung cấp các interface và class chung để làm việc với database.
- JDBC Driver là thư viện do hệ quản trị cơ sở dữ liệu cung cấp để Java giao tiếp với database đó.

Các loại JDBC Driver:

Loại	Tên	Ghi chú
Type 1	JDBC-ODBC Bridge	Cũ, không còn dùng trong Java hiện đại
Type 2	Native-API Driver	Cần thư viện native của database
Type 3	Network Protocol Driver	Giao tiếp qua một middleware server
Type 4	Thin Driver	Driver thuần Java, phổ biến nhất hiện nay

2. DriverManager

- **DriverManager** quản lý danh sách JDBC driver.
- Dùng để thiết lập kết nối tới database.
- Với JDBC 4 trở lên, driver thường được tự động nạp khi đã có trong classpath.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

String url = "jdbc:mysql://localhost:3306/java_db";
String username = "root";
String password = "your_password";

Connection conn = DriverManager.getConnection(url, username, password);
```

3. Connection

- **Connection** đại diện cho một phiên làm việc với cơ sở dữ liệu.
- Được tạo từ **DriverManager** hoặc **DataSource**.

- Cần đóng kết nối sau khi dùng xong.

```
String url = "jdbc:mysql://localhost:3306/java_db";

try (Connection conn = DriverManager.getConnection(url, "root",
"your_password")) {
    // Sử dụng kết nối tại đây
}
```

`try-with-resources` tự động gọi `close()` khi kết thúc khối lệnh.

4. Statement

- `Statement` dùng để thực thi các câu lệnh SQL đơn giản.
- Không an toàn khi ghép trực tiếp dữ liệu người dùng vào SQL vì có nguy cơ SQL Injection.
- Phù hợp với câu SQL cố định, không có tham số đầu vào.

```
String sql = "SELECT id, name FROM users";

try (Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(sql)) {
    // Đọc kết quả từ rs
}
```

5. PreparedStatement

- `PreparedStatement` là phiên bản an toàn hơn của `Statement`.
- Dùng dấu `?` làm tham số và gán giá trị qua các hàm `set...()`.
- Giúp tránh SQL Injection và thường hiệu quả hơn khi tái sử dụng câu lệnh.

```
String sql = "SELECT id, name FROM users WHERE id = ?";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 101);

    try (ResultSet rs = ps.executeQuery()) {
        // Đọc kết quả từ rs
    }
}
```

6. ResultSet

- `ResultSet` đại diện cho kết quả trả về của một truy vấn `SELECT`.
- Con trỏ ban đầu đứng trước dòng dữ liệu đầu tiên.
- Dùng `next()` để di chuyển lần lượt qua từng dòng.

```
while (rs.next()) {  
    int id = rs.getInt("id");  
    String name = rs.getString("name");  
    System.out.println(id + " - " + name);  
}
```

7. Sơ đồ quy trình CRUD

Thao tác	SQL	Ví dụ
Tạo mới (Create)	INSERT	INSERT INTO users(name, email) VALUES (?, ?)
Đọc (Read)	SELECT	SELECT * FROM users WHERE id = ?
Cập nhật (Update)	UPDATE	UPDATE users SET name = ? WHERE id = ?
Xóa (Delete)	DELETE	DELETE FROM users WHERE id = ?

- Dùng `executeQuery()` cho `SELECT` để nhận về `ResultSet`.
- Dùng `executeUpdate()` cho `INSERT`, `UPDATE`, `DELETE` để nhận số dòng bị ảnh hưởng.

```
String sql = "INSERT INTO users(name, email) VALUES (?, ?)";  
  
try (PreparedStatement ps = conn.prepareStatement(sql)) {  
    ps.setString(1, "An");  
    ps.setString(2, "an@example.com");  
  
    int rows = ps.executeUpdate();  
    System.out.println("Số dòng đã thêm: " + rows);  
}
```

8. Điểm chính

- Nạp JDBC driver (thường tự động từ JDBC 4 trở lên).
- Lấy `Connection`.
- Tạo `Statement` hoặc ưu tiên `PreparedStatement`.
- Thực thi câu truy vấn SQL.
- Xử lý `ResultSet` nếu có.
- Đóng tài nguyên.

9. Thực hành tốt

- Ưu tiên `PreparedStatement` để an toàn và hiệu quả hơn.
- Luôn đóng tài nguyên bằng `try-with-resources`.
- Không ghi mật khẩu database trực tiếp vào mã nguồn; dùng biến môi trường hoặc file cấu hình bảo mật.
- Giao dịch gồm nhiều thao tác liên quan nên dùng transaction.

10. Các lỗi thường gặp

Vấn đề	Cách tránh
Quên đóng tài nguyên	Dùng <code>try-with-resources</code>
SQL Injection	Không nối trực tiếp dữ liệu người dùng vào SQL; dùng <code>PreparedStatement</code>
Không xử lý <code>SQLException</code>	Bắt hoặc khai báo ngoại lệ và ghi log phù hợp
Thiếu JDBC driver	Thêm đúng dependency/driver của database vào project